

Chihuahua or muffin? Image lab reflection.

In this lab, I worked on the “Chihuahua or Muffin” image classification workshop using a combination of Google Colab, GitHub, Jupyter Notebook, and PyTorch. The primary objective was to build and train a neural network that could classify images into two categories whether the image was a chihuahua or muffin. The project involves cloning a GitHub repository, loading image data, building a simple neural network, training the model, and then checking how well the model performed on validation images. According to the notebook, this model uses training data to learn patterns and validation data to test whether it can make predictions on new images, not just memorize the training examples.

An important concept that I learned from this lab was how images are prepared for machine learning, since images cannot be used directly by the neural network. In order for the image to be used by the neural network, these images first need to be resized and converted into tensors, which are multi-dimensional arrays that PyTorch can process. IN the notebook, I set the width and height to 64 each, originally 5, and this is because it keeps more image detail than a very small size, while large image sizes give the model more information, such as the shape, color, texture, and facial details that may help it tell the difference between a chihuahua and a muffin. In addition, I learned that larger image sizes increase the number of input features, which could make training slower and therefore require more computing power.

A challenge I faced when completing the notebook was trying adding the missing parts into the notebook, such as the height and width, in addition to the model initialization, transforms, and batch size. At first, it was unclear about how the values play a role in the model, but after working thought the lab, I learn a critical fact that these settings affect how the data is prepared and how the model learns. Another significant challenge to the lab was understanding the training process, including the forward pass, loss function, optimizer, and epochs. However, the model was able to make prediction, calculate the chance of an error, and then updates it weights to improve the predictions.

An interesting fact about the lab was that mage size can affect model performance. This is because if the model were to use low quality images, then the model would end up losing important details. On the other hand, it's very important for models to use high quality images as those preserve more useful information, but the training process becomes more complex, proving that machine learning uses many decisions as there is not perfect answer, since different settings can change the results based on the performance of the models.

Overall, this lab was able to provide clear pictures on the relationships between neural networks and image classification. From the lab, I learned how images are prepared for a

model, how neural networks train off of these images, and how the validation data from the results can evaluate performance. Also by changing the height and width parameters, I learned that these can affect the model's accuracy as well as training speed. In conclusion, this activity helped me better understand how machine learning models learn from data and make predictions.

References.

- *Transforming images, videos, boxes and more — Torchvision 0.27 documentation.* (n.d.). <https://docs.pytorch.org/vision/stable/transforms.html>
- Contributors, P. (2023, January 1). CrossEntropyLoss. <https://docs.pytorch.org/docs/2.12/generated/torch.nn.CrossEntropyLoss.html>
- Contributors, P. (n.d.). Torch.utils.data. <https://docs.pytorch.org/docs/2.12/data.html>